RTL Design Sherpa

RTL AMBA AXI5-Lite Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXI5-Lite (AXIL5) Modules

Location: `rtl/amba/axil5/` **Sibling family:** AXI4-Lite

1.1 Overview

Sixteen modules mirroring the AXI4-Lite family one for one, with the AXI5-Lite optional signal groups threaded through. Same channel set, same handshakes, same single-beat semantics; what AXI5-Lite adds is sideband.

1.2 Scope of This Implementation

These modules **transport** AXI5-Lite signals. They do not execute AXI5-Lite semantics:

- MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. Nothing here interprets a partition ID or an encryption context.
- POISON is carried. It is never generated and never checked.
- LOCK is carried with no exclusive-access monitor behind it. A completer returning EXOKAY is not validated against anything.

Those behaviours belong to the endpoints on either side. Read this before treating the family as a full AXI5-Lite protocol stack – it is not one.

1.3 Module Categories

1.3.1 Transport (4)

Module	Channels
axil5_master_rd	AR, R
axil5_master_wr	AW, W, B
axil5_slave_rd	AR, R
axil5_slave_wr	AW, W, B

1.3.2 Clock-gated (4)

Module	Adds
axil5_master_rd_cg	One `amba_clock_gate_ctrl` over the whole inner module
axil5_master_wr_cg	ditto
axil5_slave_rd_cg	ditto
axil5_slave_wr_cg	ditto

1.3.3 Monitored (4)

Module	Adds
axil5_master_rd_mon	`axi_monitor_filtered`, monbus packet output
axil5_master_wr_mon	ditto
axil5_slave_rd_mon	ditto
axil5_slave_wr_mon	ditto

1.3.4 Monitored and clock-gated (4)

Module	Adds
axil5_master_rd_mon_cg	Both, with ready held low while gated
axil5_master_wr_mon_cg	ditto
axil5_slave_rd_mon_cg	ditto
axil5_slave_wr_mon_cg	ditto

1.4 AXI4-Lite vs AXI5-Lite in This Library

Feature	AXI4-Lite	AXI5-Lite
Channels	AW, W, B, AR, R	same
Burst support	none	none
Transaction ID	none	none
User sideband	not present	AxUSER/WUSER/BUSER/RUSER, gated by ENABLE_USER
Trace	not present	AxTRACE/BTRACE/RTRACE, gated by ENABLE_TRACE
Loopback ID	not present	AxLOOP/BLOOP/RLOOP, gated by ENABLE_LOOP
MPAM / MECID / NSAID	not present	on the address channels, each independently gated
Poison	not present	WPOISON/RPOISON, one bit per 64 data bits
Exclusive access	not present	AxLOCK, gated by ENABLE_LOCK

With every group disabled the two are the same interface. The packed payload widths match channel for channel, which is what lets one testbench bind to either family.

1.5 Quick Start

```
// AXI4-Lite-equivalent: every group off, payload widths identical to
axil4
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .ENABLE_USER (0), .ENABLE_TRACE (0), .ENABLE_LOOP
(0), .ENABLE_MPAM (0),
    .ENABLE_MECID(0), .ENABLE_NSAID(0), .ENABLE_POISON(0), .ENABLE_LOC
K (0)
) u_plain ( /* ... */ );

// With the security qualifiers, which is what AXI5-Lite is usually
for
```

```
axil5_master_rd #(
    .AXIL_ADDR_WIDTH (32),
    .AXIL_DATA_WIDTH (32),
    .NSAID_WIDTH (4), .MPAM_WIDTH (11), .MECID_WIDTH (16),
    .ENABLE_NSAID(1), .ENABLE_MPAM(1), .ENABLE_MECID(1),
    .ENABLE_USER (0), .ENABLE_TRACE(0), .ENABLE_LOOP (0),
    .ENABLE_POISON(0), .ENABLE_LOCK(0)
) u_secure ( /* ... */ );
```

1.6 Testing

`val/amba/test_axil5_*.py` – sixteen runners, 42 parameterised cases, all passing. They drive the modules with **every optional group enabled**, which makes them the only tests in the repo that exercise the AXI5-Lite sideband against real transport RTL rather than a behavioural model.

The testbench classes in `bin/TBClasses/axil5/` are the AXI4-Lite ones with the component factories swapped and the group widths set in `COMPONENT_KWARGS`. One definition of the traffic and its checks.

Also here: `test_axil5_opt_signals.py` drives `axil5_opt_slave`, a behavioural slave under `rtl/amba/axil5/test-modules/` built purely so the BFMs had a DUT carrying every optional port. It is test collateral – do not instantiate it in a design.

1.7 Known Limitations

- No protocol checking of the optional groups. The monitored variants observe handshakes, addresses, responses and timing; `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON.
- No exclusive-access monitor. `AxLOCK` is transported; nothing tracks exclusive state or validates EXOKAY.
- No poison generation or checking.
- Timing diagrams are not yet drawn for this family.

1.8 Navigation

AMBA overview | AXI4-Lite | AXI5

2 AXIL5 Master Read

Module: `axil5_master_rd.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd`

2.1 Overview

The AXIL5 Master Read provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd` with those groups threaded through the packed SKID payload.

2.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

---

2.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	
`AXIL_DATA_WIDTH`	int	32	
`USER_WIDTH`	int	4	
`LOOP_WIDTH`	int	3	
`MPAM_WIDTH`	int	11	
`MECID_WIDTH`	int	16	
`NSAID_WIDTH`	int	4	
`ENABLE_USER`	bit	1	
`ENABLE_TRACE`	bit	1	
`ENABLE_LOOP`	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
ARSize	int	AW + 3 +	
RSize	int	DW + 2 +	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

2.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
fub_araddr	Input	[AW-1:0]	
fub_arprot	Input	[2:0]	

Port	Direction	Width	Description
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_armpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_armpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	

Port	Direction	Width	Description
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
busy	Output	1	

2.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

2.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

2.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

2.5 Verification

`val/amba/test_axil5_master_rd.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

2.6 Related Modules

- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd` – the AXI4-Lite counterpart
- AXI4-Lite modules

2.7 Navigation

AXI5-Lite index | AMBA overview

3 AXIL5 Master Read, Clock-Gated

Module: `axil5_master_rd_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_cg`

3.1 Overview

The AXIL5 Master Read, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_cg` with those groups threaded through the packed SKID payload.

3.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

3.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	

Parameter	Type	Default	Description
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

3.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_araddr	Input	[AW-1:0]	
fub_arprot	Input	[2:0]	
fub_arlock	Input	1	
fub_aruser	Input	[UW-1:0]	
fub_artrace	Input	1	
fub_arloop	Input	[LW-1:0]	
fub_armpam	Input	[MW-1:0]	
fub_armecid	Input	[EW-1:0]	
fub_arnsaid	Input	[NW-1:0]	
fub_arvalid	Input	1	
fub_arready	Output	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	

Port	Direction	Width	Description
m_axil_arloop	Output	[LW-1:0]	
m_axil_armpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
fub_rdata	Output	[DW-1:0]	
fub_rresp	Output	[1:0]	
fub_ruser	Output	[UW-1:0]	
fub_rtrace	Output	1	
fub_rloop	Output	[LW-1:0]	
fub_rpoison	Output	[PW-1:0]	
fub_rvalid	Output	1	
fub_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

3.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

3.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

3.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

3.5 Verification

`val/amba/test_axil5_master_rd_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

3.6 Related Modules

- axil5_master_rd
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

3.7 Navigation

AXI5-Lite index | AMBA overview

4 AXIL5 Master Read Monitor

Module: `axil5_master_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon`

4.1 Overview

The AXIL5 Master Read Monitor provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon` with those groups threaded through the packed SKID payload.

4.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

4.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AR`	int	`2`	
`SKID_DEPTH_R`	int	`4`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	

Parameter	Type	Default	Description
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	
ID_MATCH_COUNT	int	0	
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	

Parameter	Type	Default	Description
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

4.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	

Port	Direction	Width	Description
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_araddr	Input	[AW-1:0]	
fub_axil_arprot	Input	[2:0]	
fub_axil_arlock	Input	1	
fub_axil_aruser	Input	[UW-1:0]	
fub_axil_artrace	Input	1	
fub_axil_arloop	Input	[LW-1:0]	
fub_axil_armpam	Input	[MW-1:0]	
fub_axil_armecid	Input	[EW-1:0]	
fub_axil_arnsaid	Input	[NW-1:0]	
fub_axil_arvalid	Input	1	
fub_axil_arready	Output	1	
fub_axil_rdata	Output	[DW-1:0]	
fub_axil_rresp	Output	[1:0]	
fub_axil_ruser	Output	[UW-1:0]	
fub_axil_rtrace	Output	1	
fub_axil_rloop	Output	[LW-1:0]	
fub_axil_rpoison	Output	[PW-1:0]	
fub_axil_rvalid	Output	1	
fub_axil_rready	Input	1	
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_armpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	

Port	Direction	Width	Description
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name

Port	Direction	Width	Description
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	

Port	Direction	Width	Description
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

4.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`
TRACE	`ENABLE_TRACE`	`AxTRACE`, `BTRACE`, `RTRACE`	1
LOOP	`ENABLE_LOOP`	`AxLOOP`, `BLOOP`, `RLOOP`	`LOOP_WIDTH`
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

4.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

4.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

4.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

4.6 Verification

`val/amba/test_axil5_master_rd_mon.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

4.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_rd_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

4.8 Navigation

AXI5-Lite index | AMBA overview

5 AXIL5 Master Read Monitor, Clock-Gated

Module: `axil5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_rd_mon_cg`

5.1 Overview

The AXIL5 Master Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_rd_mon_cg` with those groups threaded through the packed SKID payload.

5.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

- Monitor plus clock gating; ready signals held low while gated

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

5.3 Ports

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_araddr`	Input	`[AW-1:0]`	
`fub_axil_arprot`	Input	`[2:0]`	
`fub_axil_arlock`	Input	1	
`fub_axil_aruser`	Input	`[UW-1:0]`	
`fub_axil_artrace`	Input	1	
`fub_axil_arloop`	Input	`[LW-1:0]`	
`fub_axil_armpam`	Input	`[MW-1:0]`	
`fub_axil_armecid`	Input	`[EW-1:0]`	
`fub_axil_arnsaid`	Input	`[NW-1:0]`	
`fub_axil_arvalid`	Input	1	
`fub_axil_arready`	Output	1	
`fub_axil_rdata`	Output	`[DW-1:0]`	
`fub_axil_rresp`	Output	`[1:0]`	
`fub_axil_ruser`	Output	`[UW-1:0]`	
`fub_axil_rtrace`	Output	1	
`fub_axil_rloop`	Output	`[LW-1:0]`	
`fub_axil_rpoison`	Output	`[PW-1:0]`	
`fub_axil_rvalid`	Output	1	
`fub_axil_rready`	Input	1	

Port	Direction	Width	Description
m_axil_araddr	Output	[AW-1:0]	
m_axil_arprot	Output	[2:0]	
m_axil_arlock	Output	1	
m_axil_aruser	Output	[UW-1:0]	
m_axil_artrace	Output	1	
m_axil_arloop	Output	[LW-1:0]	
m_axil_armpam	Output	[MW-1:0]	
m_axil_armecid	Output	[EW-1:0]	
m_axil_arnsaid	Output	[NW-1:0]	
m_axil_arvalid	Output	1	
m_axil_arready	Input	1	
m_axil_rdata	Input	[DW-1:0]	
m_axil_rresp	Input	[1:0]	
m_axil_ruser	Input	[UW-1:0]	
m_axil_rtrace	Input	1	
m_axil_rloop	Input	[LW-1:0]	
m_axil_rpoison	Input	[PW-1:0]	
m_axil_rvalid	Input	1	
m_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets

Port	Direction	Width	Description
`cfg_threshold_enable`	Input	1	Enable threshold packets
`cfg_debug_enable`	Input	1	Enable debug packets
`cfg_timeout_cycles`	Input	`[15:0]`	Timeout threshold in MICROSECONDS (1 us tick), despite the name
`cfg_freq_sel`	Input	`[3:0]`	counter_freq_invariant LUT index
`cfg_latency_threshold`	Input	`[31:0]`	Latency threshold for alerts
`cfg_axi_pkt_mask`	Input	`[15:0]`	Drop mask for packet types
`cfg_axi_err_select`	Input	`[15:0]`	Error select for packet types
`cfg_axi_error_mask`	Input	`[15:0]`	Individual error event mask
`cfg_axi_timeout_mask`	Input	`[15:0]`	Individual timeout event mask
`cfg_axi_compl_mask`	Input	`[15:0]`	Individual completion event mask
`cfg_axi_thresh_mask`	Input	`[15:0]`	Individual threshold event mask
`cfg_axi_perf_mask`	Input	`[15:0]`	Individual performance event mask
`cfg_axi_addr_mask`	Input	`[15:0]`	Individual address match event mask

Port	Direction	Width	Description
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	

Port	Direction	Width	Description
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

5.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits

Group	Parameter	Signals on this module	Width
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

5.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

5.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

5.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

ACLK_MHZ is not decoration. It builds the microsecond tick LUT in counter_freq_invariant. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

NUM_BANKS > 1 on a WRITE monitor requires USE_WDATA_ORDER_Q = 1. axi_monitor_trans_mgr fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets ADDR_FILTER_ENABLE but leaves cfg_addr_filter_enable low filters nothing and looks broken. The parameter and the runtime port are both required.

5.6 Verification

val/amba/test_axil5_master_rd_mon_cg.py drives this module with the AXI5-Lite BFMs and **every optional group enabled** – TBClasses/axil5 sets the group widths in COMPONENT_KWARGS to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

5.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_wr
- axil5_master_wr_cg
- axil5_master_wr_mon
- axil4_master_rd_mon_cg – the AXI4-Lite counterpart
- AXI4-Lite modules

5.8 Navigation

AXI5-Lite index | AMBA overview

6 AXIL5 Master Write

Module: `axil5_master_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr`

6.1 Overview

The AXIL5 Master Write provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr` with those groups threaded through the packed SKID payload.

6.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

---

6.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	
`AXIL_DATA_WIDTH`	int	32	
`USER_WIDTH`	int	4	
`LOOP_WIDTH`	int	3	
`MPAM_WIDTH`	int	11	
`MECID_WIDTH`	int	16	
`NSAID_WIDTH`	int	4	
`ENABLE_USER`	bit	1	
`ENABLE_TRACE`	bit	1	
`ENABLE_LOOP`	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	
BSize	int	2 +	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

6.3 Ports

Port	Direction	Width	Description
aclk	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	
fub_awmecid	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
fub_wdata	Input	[DW-1:0]	
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	

Port	Direction	Width	Description
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
busy	Output	1	

6.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1

Group	Parameter	Signals on this module	Width
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

6.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

6.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

6.5 Verification

`val/amba/test_axil5_master_wr.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

6.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr_cg
- axil5_master_wr_mon
- `axil4_master_wr` – the AXI4-Lite counterpart
- AXI4-Lite modules

6.7 Navigation

AXI5-Lite index | AMBA overview

7 AXIL5 Master Write, Clock-Gated

Module: `axil5_master_wr_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_cg`

7.1 Overview

The AXIL5 Master Write, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_cg` with those groups threaded through the packed SKID payload.

7.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

7.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	
`AXIL_DATA_WIDTH`	int	32	
`USER_WIDTH`	int	4	
`LOOP_WIDTH`	int	3	
`MPAM_WIDTH`	int	11	
`MECID_WIDTH`	int	16	
`NSAID_WIDTH`	int	4	
`ENABLE_USER`	bit	1	
`ENABLE_TRACE`	bit	1	
`ENABLE_LOOP`	bit	1	
`ENABLE_MPAM`	bit	1	
`ENABLE_MECID`	bit	1	
`ENABLE_NSAID`	bit	1	
`ENABLE_POISON`	bit	1	
`ENABLE_LOCK`	bit	1	
`SKID_DEPTH_AW`	int	2	

Parameter	Type	Default	Description
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

7.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
fub_awaddr	Input	[AW-1:0]	
fub_awprot	Input	[2:0]	
fub_awlock	Input	1	

Port	Direction	Width	Description
fub_awuser	Input	[UW-1:0]	
fub_awtrace	Input	1	
fub_awloop	Input	[LW-1:0]	
fub_awmpam	Input	[MW-1:0]	
fub_awmecid	Input	[EW-1:0]	
fub_awnsaid	Input	[NW-1:0]	
fub_awvalid	Input	1	
fub_awready	Output	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
fub_wdata	Input	[DW-1:0]	
fub_wstrb	Input	[SW-1:0]	
fub_wuser	Input	[UW-1:0]	
fub_wpoison	Input	[PW-1:0]	
fub_wvalid	Input	1	
fub_wready	Output	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[SW-1:0]	
m_axil_wuser	Output	[UW-1:0]	

Port	Direction	Width	Description
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
fub_bresp	Output	[1:0]	
fub_buser	Output	[UW-1:0]	
fub_btrace	Output	1	
fub_bloop	Output	[LW-1:0]	
fub_bvalid	Output	1	
fub_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

7.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1

Group	Parameter	Signals on this module	Width
LOOP	`ENABLE_LOOP`	`AxLOOP`, `BLOOP`, `RLOOP`	`LOOP_WIDTH`
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

7.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

7.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

7.5 Verification

`val/amba/test_axil5_master_wr_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

7.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_mon
- `axil4_master_wr_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

7.7 Navigation

AXI5-Lite index | AMBA overview

8 AXIL5 Master Write Monitor

Module: `axil5_master_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon`

8.1 Overview

The AXIL5 Master Write Monitor provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon` with those groups threaded through the packed SKID payload.

8.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

8.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	
`CFI_MAX_FREQ_MHZ`	int	`ACLK_MHZ`	
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor, tie outputs
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`MAX_TRANSACTIONS`	int	`8`	Maximum outstanding transactions (reduced for AXIL)
`USE_WDATA_ORDER_Q`	bit	`1'b0`	

Parameter	Type	Default	Description
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	
ID_MATCH_COUNT	int	0	
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	

Parameter	Type	Default	Description
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

8.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
fub_axil_awaddr	Input	[AW-1:0]	
fub_axil_awprot	Input	[2:0]	
fub_axil_awlock	Input	1	
fub_axil_awuser	Input	[UW-1:0]	
fub_axil_awtrace	Input	1	
fub_axil_awloop	Input	[LW-1:0]	
fub_axil_awmpam	Input	[MW-1:0]	
fub_axil_awmecid	Input	[EW-1:0]	

Port	Direction	Width	Description
fub_axil_awnsaid	Input	[NW-1:0]	
fub_axil_awvalid	Input	1	
fub_axil_awready	Output	1	
fub_axil_wdata	Input	[DW-1:0]	
fub_axil_wstrb	Input	[DW/8-1:0]	
fub_axil_wuser	Input	[UW-1:0]	
fub_axil_wpoison	Input	[PW-1:0]	
fub_axil_wvalid	Input	1	
fub_axil_wready	Output	1	
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	

Port	Direction	Width	Description
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts

Port	Direction	Width	Description
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid

Port	Direction	Width	Description
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

8.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`
TRACE	`ENABLE_TRACE`	`AxTRACE`, `BTRACE`, `RTRACE`	1
LOOP	`ENABLE_LOOP`	`AxLOOP`, `BLOOP`, `RLOOP`	`LOOP_WIDTH`
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

8.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

8.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

8.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

8.6 Verification

`val/amba/test_axil5_master_wr_mon.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

8.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_master_wr_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

8.8 Navigation

AXI5-Lite index | AMBA overview

9 AXIL5 Master Write Monitor, Clock-Gated

Module: `axil5_master_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_master_wr_mon_cg`

9.1 Overview

The AXIL5 Master Write Monitor, Clock-Gated provides a buffered AXI5-Lite write interface for master devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_master_wr_mon_cg` with those groups threaded through the packed SKID payload.

9.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

- Monitor plus clock gating; ready signals held low while gated

9.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	2	
SKID_DEPTH_B	int	2	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ?	

Parameter	Type	Default	Description
		`(DW / 64) : 1`	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

9.3 Ports

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`fub_axil_awaddr`	Input	`[AW-1:0]`	
`fub_axil_awprot`	Input	`[2:0]`	
`fub_axil_awlock`	Input	1	
`fub_axil_awuser`	Input	`[UW-1:0]`	
`fub_axil_awtrace`	Input	1	
`fub_axil_awloop`	Input	`[LW-1:0]`	
`fub_axil_awmpam`	Input	`[MW-1:0]`	
`fub_axil_awmecid`	Input	`[EW-1:0]`	
`fub_axil_awnsaid`	Input	`[NW-1:0]`	
`fub_axil_awvalid`	Input	1	
`fub_axil_awready`	Output	1	
`fub_axil_wdata`	Input	`[DW-1:0]`	
`fub_axil_wstrb`	Input	`[DW/8-1:0]`	
`fub_axil_wuser`	Input	`[UW-1:0]`	
`fub_axil_wpoison`	Input	`[PW-1:0]`	
`fub_axil_wvalid`	Input	1	
`fub_axil_wready`	Output	1	

Port	Direction	Width	Description
fub_axil_bresp	Output	[1:0]	
fub_axil_buser	Output	[UW-1:0]	
fub_axil_btrace	Output	1	
fub_axil_bloop	Output	[LW-1:0]	
fub_axil_bvalid	Output	1	
fub_axil_bready	Input	1	
m_axil_awaddr	Output	[AW-1:0]	
m_axil_awprot	Output	[2:0]	
m_axil_awlock	Output	1	
m_axil_awuser	Output	[UW-1:0]	
m_axil_awtrace	Output	1	
m_axil_awloop	Output	[LW-1:0]	
m_axil_awmpam	Output	[MW-1:0]	
m_axil_awmecid	Output	[EW-1:0]	
m_axil_awnsaid	Output	[NW-1:0]	
m_axil_awvalid	Output	1	
m_axil_awready	Input	1	
m_axil_wdata	Output	[DW-1:0]	
m_axil_wstrb	Output	[DW/8-1:0]	
m_axil_wuser	Output	[UW-1:0]	
m_axil_wpoison	Output	[PW-1:0]	
m_axil_wvalid	Output	1	
m_axil_wready	Input	1	
m_axil_bresp	Input	[1:0]	
m_axil_buser	Input	[UW-1:0]	
m_axil_btrace	Input	1	
m_axil_bloop	Input	[LW-1:0]	
m_axil_bvalid	Input	1	
m_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable

Port	Direction	Width	Description
			monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_ma	Input	[15:0]	Individual

Port	Direction	Width	Description
sk			completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count

Port	Direction	Width	Description
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

9.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`
TRACE	`ENABLE_TRACE`	`AxTRACE`, `BTRACE`, `RTRACE`	1
LOOP	`ENABLE_LOOP`	`AxLOOP`, `BLOOP`, `RLOOP`	`LOOP_WIDTH`
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

9.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

9.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

9.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

9.6 Verification

`val/amba/test_axil5_master_wr_mon_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

9.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon

- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_master_wr_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

9.8 Navigation

AXI5-Lite index | AMBA overview

10 AXIL5 Slave Read

Module: `axil5_slave_rd.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd`

10.1 Overview

The AXIL5 Slave Read provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd` with those groups threaded through the packed SKID payload.

10.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

---

10.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	

Parameter	Type	Default	Description
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
ARSize	int	AW + 3 +	
RSize	int	DW + 2 +	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

10.3 Ports

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`s_axil_araddr`	Input	`[AW-1:0]`	
`s_axil_arprot`	Input	`[2:0]`	
`s_axil_arlock`	Input	1	
`s_axil_aruser`	Input	`[UW-1:0]`	
`s_axil_artrace`	Input	1	
`s_axil_arloop`	Input	`[LW-1:0]`	
`s_axil_armpam`	Input	`[MW-1:0]`	
`s_axil_armecid`	Input	`[EW-1:0]`	
`s_axil_arnsaid`	Input	`[NW-1:0]`	
`s_axil_arvalid`	Input	1	
`s_axil_arready`	Output	1	
`fub_araddr`	Output	`[AW-1:0]`	
`fub_arprot`	Output	`[2:0]`	
`fub_arlock`	Output	1	
`fub_aruser`	Output	`[UW-1:0]`	
`fub_artrace`	Output	1	
`fub_arloop`	Output	`[LW-1:0]`	
`fub_armpam`	Output	`[MW-1:0]`	
`fub_armecid`	Output	`[EW-1:0]`	
`fub_arnsaid`	Output	`[NW-1:0]`	
`fub_arvalid`	Output	1	

Port	Direction	Width	Description
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
busy	Output	1	

10.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH

Group	Parameter	Signals on this module	Width
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

10.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

10.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

10.5 Verification

`val/amba/test_axil5_slave_rd.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

10.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd` – the AXI4-Lite counterpart
- AXI4-Lite modules

10.7 Navigation

AXI5-Lite index | AMBA overview

11 AXIL5 Slave Read, Clock-Gated

Module: `axil5_slave_rd_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_cg`

11.1 Overview

The AXIL5 Slave Read, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_cg` with those groups threaded through the packed SKID payload.

11.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

11.2 Parameters

Parameter	Type	Default	Description
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AR	int	2	

Parameter	Type	Default	Description
SKID_DEPTH_R	int	4	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

11.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	

Port	Direction	Width	Description
s_axil_arloop	Input	[LW-1:0]	
s_axil_armpam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
fub_araddr	Output	[AW-1:0]	
fub_arprot	Output	[2:0]	
fub_arlock	Output	1	
fub_aruser	Output	[UW-1:0]	
fub_artrace	Output	1	
fub_arloop	Output	[LW-1:0]	
fub_armpam	Output	[MW-1:0]	
fub_armecid	Output	[EW-1:0]	
fub_arnsaid	Output	[NW-1:0]	
fub_arvalid	Output	1	
fub_arready	Input	1	
fub_rdata	Input	[DW-1:0]	
fub_rresp	Input	[1:0]	
fub_ruser	Input	[UW-1:0]	
fub_rtrace	Input	1	
fub_rloop	Input	[LW-1:0]	
fub_rpoison	Input	[PW-1:0]	
fub_rvalid	Input	1	
fub_rready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	

Port	Direction	Width	Description
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

11.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

11.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

11.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

11.5 Verification

`val/amba/test_axil5_slave_rd_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

11.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

11.7 Navigation

AXI5-Lite index | AMBA overview

12 AXIL5 Slave Read Monitor

Module: `axil5_slave_rd_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon`

12.1 Overview

The AXIL5 Slave Read Monitor provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon` with those groups threaded through the packed SKID payload.

12.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

12.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	
ID_MATCH_COUNT	int	0	
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	

Parameter	Type	Default	Description
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

12.3 Ports

Port	Direction	Width	Description
`aclk`	Input	1	
`aresetn`	Input	1	
`cam_clear`	Input	1	sync clear of the monitor trans CAM
`s_axil_araddr`	Input	`[AW-1:0]`	
`s_axil_arprot`	Input	`[2:0]`	
`s_axil_arlock`	Input	1	
`s_axil_aruser`	Input	`[UW-1:0]`	
`s_axil_artrace`	Input	1	
`s_axil_arloop`	Input	`[LW-1:0]`	
`s_axil_armpam`	Input	`[MW-1:0]`	
`s_axil_armecid`	Input	`[EW-1:0]`	
`s_axil_arnsaid`	Input	`[NW-1:0]`	
`s_axil_arvalid`	Input	1	
`s_axil_arready`	Output	1	
`s_axil_rdata`	Output	`[DW-1:0]`	
`s_axil_rresp`	Output	`[1:0]`	
`s_axil_ruser`	Output	`[UW-1:0]`	
`s_axil_rtrace`	Output	1	
`s_axil_rloop`	Output	`[LW-1:0]`	
`s_axil_rpoison`	Output	`[PW-1:0]`	
`s_axil_rvalid`	Output	1	
`s_axil_rready`	Input	1	

Port	Direction	Width	Description
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_armpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion packets

Port	Direction	Width	Description
`cfg_threshold_enable`	Input	1	Enable threshold packets
`cfg_debug_enable`	Input	1	Enable debug packets
`cfg_timeout_cycles`	Input	`[15:0]`	Timeout threshold in MICROSECONDS (1 us tick), despite the name
`cfg_freq_sel`	Input	`[3:0]`	counter_freq_invariant LUT index
`cfg_latency_threshold`	Input	`[31:0]`	Latency threshold for alerts
`cfg_axi_pkt_mask`	Input	`[15:0]`	Drop mask for packet types
`cfg_axi_err_select`	Input	`[15:0]`	Error select for packet types
`cfg_axi_error_mask`	Input	`[15:0]`	Individual error event mask
`cfg_axi_timeout_mask`	Input	`[15:0]`	Individual timeout event mask
`cfg_axi_compl_mask`	Input	`[15:0]`	Individual completion event mask
`cfg_axi_thresh_mask`	Input	`[15:0]`	Individual threshold event mask
`cfg_axi_perf_mask`	Input	`[15:0]`	Individual performance event mask
`cfg_axi_addr_mask`	Input	`[15:0]`	Individual address match event mask

Port	Direction	Width	Description
`cfg_axi_debug_mask`	Input	`[15:0]`	Individual debug event mask
`cfg_addr_check_enable`	Input	1	
`cfg_addr_range_enable`	Input	`[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]`	
`cfg_addr_filter_enable`	Input	1	
`cfg_addr_filter_low`	Input	`[AW-1:0]`	
`cfg_addr_filter_high`	Input	`[AW-1:0]`	
`monbus_valid`	Output	1	Monitor bus valid
`monbus_ready`	Input	1	Monitor bus ready
`busy`	Output	1	
`active_transactions`	Output	`[7:0]`	Number of active transactions
`error_count`	Output	`[15:0]`	Total error count
`transaction_count`	Output	`[31:0]`	Total transaction count
`debug_block_ready`	Output	1	
`cfg_conflict_error`	Output	1	Configuration conflict detected
`cfg_start_event_sel`	Input	`[2:0]`	
`cfg_end_event_sel`	Input	`[2:0]`	
`cfg_start_trigger`	Input	1	
`cfg_end_trigger`	Input	1	
`cfg_window_force_close`	Input	1	
`window_active`	Output	1	
`window_cycles`	Output	`[31:0]`	

Port	Direction	Width	Description
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

12.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

12.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

12.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

12.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

12.6 Verification

`val/amba/test_axil5_slave_rd_mon.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

12.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

12.8 Navigation

AXI5-Lite index | AMBA overview

13 AXIL5 Slave Read Monitor, Clock-Gated

Module: `axil5_slave_rd_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_rd_mon_cg`

13.1 Overview

The AXIL5 Slave Read Monitor, Clock-Gated provides a buffered AXI5-Lite read interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_rd_mon_cg` with those groups threaded through the packed SKID payload.

13.1.1 Key Features

- AXI5-Lite read path (AR and R channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

---

13.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	
SKID_DEPTH_R	int	4	
AXIL_ADDR_WIDTH	int	32	
AXIL_DATA_WIDTH	int	32	
USE_MONITOR	bit	1'b1	0 = omit monitor in inner mon; outputs tied
N_ADDR_RANGES	int	0	0 = address-range checker disabled
ADDR_FILTER_ENABLE	bit	1'b0	0 = address filter inert
ACLK_MHZ	int	100	
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum

Parameter	Type	Default	Description
			outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	

Parameter	Type	Default	Description
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

13.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_araddr	Input	[AW-1:0]	
s_axil_arprot	Input	[2:0]	
s_axil_arlock	Input	1	
s_axil_aruser	Input	[UW-1:0]	
s_axil_artrace	Input	1	
s_axil_arloop	Input	[LW-1:0]	
s_axil_armpam	Input	[MW-1:0]	
s_axil_armecid	Input	[EW-1:0]	
s_axil_arnsaid	Input	[NW-1:0]	

Port	Direction	Width	Description
s_axil_arvalid	Input	1	
s_axil_arready	Output	1	
s_axil_rdata	Output	[DW-1:0]	
s_axil_rresp	Output	[1:0]	
s_axil_ruser	Output	[UW-1:0]	
s_axil_rtrace	Output	1	
s_axil_rloop	Output	[LW-1:0]	
s_axil_rpoison	Output	[PW-1:0]	
s_axil_rvalid	Output	1	
s_axil_rready	Input	1	
fub_axil_araddr	Output	[AW-1:0]	
fub_axil_arprot	Output	[2:0]	
fub_axil_arlock	Output	1	
fub_axil_aruser	Output	[UW-1:0]	
fub_axil_artrace	Output	1	
fub_axil_arloop	Output	[LW-1:0]	
fub_axil_armpam	Output	[MW-1:0]	
fub_axil_armecid	Output	[EW-1:0]	
fub_axil_arnsaid	Output	[NW-1:0]	
fub_axil_arvalid	Output	1	
fub_axil_arready	Input	1	
fub_axil_rdata	Input	[DW-1:0]	
fub_axil_rresp	Input	[1:0]	
fub_axil_ruser	Input	[UW-1:0]	
fub_axil_rtrace	Input	1	
fub_axil_rloop	Input	[LW-1:0]	
fub_axil_rpoison	Input	[PW-1:0]	
fub_axil_rvalid	Input	1	
fub_axil_rready	Output	1	
cfg_monitor_enable	Input	1	Enable

Port	Direction	Width	Description
			monitoring
`cfg_error_enable`	Input	1	Enable error detection
`cfg_timeout_enable`	Input	1	Enable timeout detection
`cfg_perf_enable`	Input	1	Enable performance monitoring
`cfg_compl_enable`	Input	1	Enable completion packets
`cfg_threshold_enable`	Input	1	Enable threshold packets
`cfg_debug_enable`	Input	1	Enable debug packets
`cfg_timeout_cycles`	Input	`[15:0]`	Timeout threshold in MICROSECONDS (1 us tick), despite the name
`cfg_freq_sel`	Input	`[3:0]`	counter_freq_invariant LUT index
`cfg_latency_threshold`	Input	`[31:0]`	Latency threshold for alerts
`cfg_axi_pkt_mask`	Input	`[15:0]`	Drop mask for packet types
`cfg_axi_err_select`	Input	`[15:0]`	Error select for packet types
`cfg_axi_error_mask`	Input	`[15:0]`	Individual error event mask
`cfg_axi_timeout_mask`	Input	`[15:0]`	Individual timeout event mask
`cfg_axi_compl_ma`	Input	`[15:0]`	Individual

Port	Direction	Width	Description
sk			completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count

Port	Direction	Width	Description
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

13.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own `ENABLE_*` parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	`ENABLE_USER`	`AxUSER`, `WUSER`, `BUSER`, `RUSER`	`USER_WIDTH`
TRACE	`ENABLE_TRACE`	`AxTRACE`, `BTRACE`, `RTRACE`	1
LOOP	`ENABLE_LOOP`	`AxLOOP`, `BLOOP`, `RLOOP`	`LOOP_WIDTH`
MPAM	`ENABLE_MPAM`	`AxMPAM`	`MPAM_WIDTH`
MECID	`ENABLE_MECID`	`AxMECID`	`MECID_WIDTH`
NSAID	`ENABLE_NSAID`	`AxNSAID`	`NSAID_WIDTH`
POISON	`ENABLE_POISON`	`WPOISON`, `RPOISON`	one bit per 64 data bits
LOCK	`ENABLE_LOCK`	`AxLOCK`	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

13.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

13.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it.

Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

13.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

13.6 Verification

`val/amba/test_axil5_slave_rd_mon_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

13.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon

- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_rd_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

13.8 Navigation

AXI5-Lite index | AMBA overview

14 AXIL5 Slave Write

Module: `axil5_slave_wr.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr`

14.1 Overview

The AXIL5 Slave Write provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr` with those groups threaded through the packed SKID payload.

14.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled

---

14.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	

Parameter	Type	Default	Description
AXIL_DATA_WIDTH	int	32	
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	
AWSize	int	AW + 3 +	
WSize	int	DW + SW +	

Parameter	Type	Default	Description
BSize	int	2 +	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how test_axil5_master_wr failed until d6266344 removed those overrides.

14.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awlock	Output	1	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awloop	Output	[LW-1:0]	

Port	Direction	Width	Description
fub_awmpam	Output	[MW-1:0]	
fub_awmecid	Output	[EW-1:0]	
fub_awnsaid	Output	[NW-1:0]	
fub_awvalid	Output	1	
fub_awready	Input	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
busy	Output	1	

14.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

14.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

14.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

14.5 Verification

`val/amba/test_axil5_slave_wr.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

14.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr` – the AXI4-Lite counterpart
- AXI4-Lite modules

14.7 Navigation

AXI5-Lite index | AMBA overview

15 AXIL5 Slave Write, Clock-Gated

Module: `axil5_slave_wr_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_cg`

15.1 Overview

The AXIL5 Slave Write, Clock-Gated provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_cg` with those groups threaded through the packed SKID payload.

15.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- One `amba_clock_gate_ctrl` gating the whole inner module

15.2 Parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	32	
`AXIL_DATA_WIDTH`	int	32	
`USER_WIDTH`	int	4	
`LOOP_WIDTH`	int	3	
`MPAM_WIDTH`	int	11	

Parameter	Type	Default	Description
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
SKID_DEPTH_AW	int	2	
SKID_DEPTH_W	int	4	
SKID_DEPTH_B	int	2	
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
SW	int	DW / 8	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

15.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cfg_cg_enable	Input	1	
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
fub_awaddr	Output	[AW-1:0]	
fub_awprot	Output	[2:0]	
fub_awlock	Output	1	
fub_awuser	Output	[UW-1:0]	
fub_awtrace	Output	1	
fub_awloop	Output	[LW-1:0]	
fub_awmpam	Output	[MW-1:0]	
fub_awmecid	Output	[EW-1:0]	
fub_awnsaid	Output	[NW-1:0]	
fub_awvalid	Output	1	
fub_awready	Input	1	

Port	Direction	Width	Description
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[SW-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
fub_wdata	Output	[DW-1:0]	
fub_wstrb	Output	[SW-1:0]	
fub_wuser	Output	[UW-1:0]	
fub_wpoison	Output	[PW-1:0]	
fub_wvalid	Output	1	
fub_wready	Input	1	
fub_bresp	Input	[1:0]	
fub_buser	Input	[UW-1:0]	
fub_btrace	Input	1	
fub_bloop	Input	[LW-1:0]	
fub_bvalid	Input	1	
fub_bready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
cg_gating	Output	1	Active gating indicator
cg_idle	Output	1	All buffers empty indicator

15.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

15.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
ARSize	35	35	75
RSize	34	34	43
AWSize	35	35	75
WSize	36	36	41
BSize	2	2	10

(at AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32 and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

15.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

15.5 Verification

`val/amba/test_axil5_slave_wr_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

15.6 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

15.7 Navigation

AXI5-Lite index | AMBA overview

16 AXIL5 Slave Write Monitor

Module: `axil5_slave_wr_mon.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon`

16.1 Overview

The AXIL5 Slave Write Monitor provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon` with those groups threaded through the packed SKID payload.

16.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets

16.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	

Parameter	Type	Default	Description
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_MONITOR	bit	1'b1	0 = omit monitor, tie outputs
N_ADDR_RANGES	int	0	0 = address-range checker disabled
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
ID_FILTER_ENABLE	bit	1'b0	
ADDR_FILTER_ENABLE	bit	1'b0	
ID_MATCH_BASE	int	0	
ID_MATCH_COUNT	int	0	
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS / 2	
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	

Parameter	Type	Default	Description
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (`AW`, `DW`, `UW`, … and the `*Size` payload widths) are computed from the ones above. **Do not override them.** Forcing a `*Size` to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until `d6266344` removed those overrides.

16.3 Ports

Port	Direction	Width	Description
aclk	Input	1	

Port	Direction	Width	Description
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	
s_axil_awloop	Input	[LW-1:0]	
s_axil_awmpam	Input	[MW-1:0]	
s_axil_awmecid	Input	[EW-1:0]	
s_axil_awnsaid	Input	[NW-1:0]	
s_axil_awvalid	Input	1	
s_axil_awready	Output	1	
s_axil_wdata	Input	[DW-1:0]	
s_axil_wstrb	Input	[DW/8-1:0]	
s_axil_wuser	Input	[UW-1:0]	
s_axil_wpoison	Input	[PW-1:0]	
s_axil_wvalid	Input	1	
s_axil_wready	Output	1	
s_axil_bresp	Output	[1:0]	
s_axil_buser	Output	[UW-1:0]	
s_axil_btrace	Output	1	
s_axil_bloop	Output	[LW-1:0]	
s_axil_bvalid	Output	1	
s_axil_bready	Input	1	
fub_axil_awaddr	Output	[AW-1:0]	
fub_axil_awprot	Output	[2:0]	
fub_axil_awlock	Output	1	

Port	Direction	Width	Description
fub_axil_awuser	Output	[UW-1:0]	
fub_axil_awtrace	Output	1	
fub_axil_awloop	Output	[LW-1:0]	
fub_axil_awmpam	Output	[MW-1:0]	
fub_axil_awmecid	Output	[EW-1:0]	
fub_axil_awnsaid	Output	[NW-1:0]	
fub_axil_awvalid	Output	1	
fub_axil_awready	Input	1	
fub_axil_wdata	Output	[DW-1:0]	
fub_axil_wstrb	Output	[DW/8-1:0]	
fub_axil_wuser	Output	[UW-1:0]	
fub_axil_wpoison	Output	[PW-1:0]	
fub_axil_wvalid	Output	1	
fub_axil_wready	Input	1	
fub_axil_bresp	Input	[1:0]	
fub_axil_buser	Input	[UW-1:0]	
fub_axil_btrace	Input	1	
fub_axil_bloop	Input	[LW-1:0]	
fub_axil_bvalid	Input	1	
fub_axil_bready	Output	1	
cfg_monitor_enable	Input	1	Enable monitoring
cfg_error_enable	Input	1	Enable error detection
cfg_timeout_enable	Input	1	Enable timeout detection
cfg_perf_enable	Input	1	Enable performance monitoring
cfg_compl_enable	Input	1	Enable completion

Port	Direction	Width	Description
			packets
cfg_threshold_enable	Input	1	Enable threshold packets
cfg_debug_enable	Input	1	Enable debug packets
cfg_timeout_cycles	Input	[15:0]	Timeout threshold in MICROSECONDS (1 us tick), despite the name
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match

Port	Direction	Width	Description
			event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_addr_check_enable	Input	1	
cfg_addr_range_enable	Input	[(N_ADDR_RANGES > 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
debug_block_ready	Output	1	
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	

Port	Direction	Width	Description
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

16.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

16.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

16.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

16.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

16.6 Verification

`val/amba/test_axil5_slave_wr_mon.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

16.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_mon` – the AXI4-Lite counterpart
- AXI4-Lite modules

16.8 Navigation

AXI5-Lite index | AMBA overview

17 AXIL5 Slave Write Monitor, Clock-Gated

Module: `axil5_slave_wr_mon_cg.sv` **Location:** `rtl/amba/axil5/` **AXI4-Lite counterpart:** `axil4_slave_wr_mon_cg`

17.1 Overview

The AXIL5 Slave Write Monitor, Clock-Gated provides a buffered AXI5-Lite write interface for slave devices.

AXI5-Lite is AXI4-Lite plus optional signal groups. It changes no channel's handshake, ordering or response semantics, so this module is structurally `axil4_slave_wr_mon_cg` with those groups threaded through the packed SKID payload.

17.1.1 Key Features

- AXI5-Lite write path (AW, W and B channels)
- Single-beat transactions: AXI-Lite has no burst support
- Configurable skid buffers on every channel for timing closure
- Eight optional signal groups, each independently enabled
- Bit-identical to the AXI4-Lite module of the same name with all groups disabled
- Integrated transaction monitor emitting monbus packets
- Monitor plus clock gating; ready signals held low while gated

17.2 Parameters

Parameter	Type	Default	Description
`SKID_DEPTH_AW`	int	`2`	
`SKID_DEPTH_W`	int	`2`	
`SKID_DEPTH_B`	int	`2`	
`AXIL_ADDR_WIDTH`	int	`32`	
`AXIL_DATA_WIDTH`	int	`32`	
`USE_MONITOR`	bit	`1'b1`	0 = omit monitor in inner mon; outputs tied
`N_ADDR_RANGES`	int	`0`	0 = address-range checker disabled
`ADDR_FILTER_ENABLE`	bit	`1'b0`	0 = address filter inert
`ACLK_MHZ`	int	`100`	
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	

Parameter	Type	Default	Description
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	
USE_WDATA_ORDER_Q	bit	1'b0	
NUM_BANKS	int	1	
MAX_TRANSACTIONS	int	8	Maximum outstanding transactions (reduced for AXIL)
ENABLE_FILTERING	bit	1	Enable packet filtering
ADD_PIPELINE_STAGE	bit	0	Add register stage for timing closure
ENABLE_ERROR_LOGIC	bit	1'b1	
ENABLE_TIMEOUT_LOGIC	bit	1'b1	
ENABLE_COMPL_LOGIC	bit	1'b1	
ENABLE_THRESHOLD_LOGIC	bit	1'b1	
ENABLE_PERF_LOGIC	bit	1'b1	
ENABLE_DEBUG_LOGIC	bit	1'b0	
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle countdown
USER_WIDTH	int	4	
LOOP_WIDTH	int	3	
MPAM_WIDTH	int	11	
MECID_WIDTH	int	16	
NSAID_WIDTH	int	4	
ENABLE_USER	bit	1	
ENABLE_TRACE	bit	1	
ENABLE_LOOP	bit	1	

Parameter	Type	Default	Description
ENABLE_MPAM	bit	1	
ENABLE_MECID	bit	1	
ENABLE_NSAID	bit	1	
ENABLE_POISON	bit	1	
ENABLE_LOCK	bit	1	
AW	int	AXIL_ADDR_WIDTH	
DW	int	AXIL_DATA_WIDTH	
UW	int	USER_WIDTH	
LW	int	LOOP_WIDTH	
MW	int	MPAM_WIDTH	
EW	int	MECID_WIDTH	
NW	int	NSAID_WIDTH	
PW	int	(DW / 64) > 0 ? (DW / 64) : 1	

The derived parameters (AW, DW, UW, … and the *Size payload widths) are computed from the ones above. **Do not override them.** Forcing a *Size to a value the RTL did not derive makes every optional-group field a part-select past the end of the vector – which is exactly how `test_axil5_master_wr` failed until d6266344 removed those overrides.

17.3 Ports

Port	Direction	Width	Description
aclk	Input	1	
aresetn	Input	1	
cam_clear	Input	1	sync clear of the monitor trans CAM
s_axil_awaddr	Input	[AW-1:0]	
s_axil_awprot	Input	[2:0]	
s_axil_awlock	Input	1	
s_axil_awuser	Input	[UW-1:0]	
s_axil_awtrace	Input	1	

Port	Direction	Width	Description
`s_axil_awloop`	Input	`[LW-1:0]`	
`s_axil_awmpam`	Input	`[MW-1:0]`	
`s_axil_awmecid`	Input	`[EW-1:0]`	
`s_axil_awnsaid`	Input	`[NW-1:0]`	
`s_axil_awvalid`	Input	1	
`s_axil_awready`	Output	1	
`s_axil_wdata`	Input	`[DW-1:0]`	
`s_axil_wstrb`	Input	`[DW/8-1:0]`	
`s_axil_wuser`	Input	`[UW-1:0]`	
`s_axil_wpoison`	Input	`[PW-1:0]`	
`s_axil_wvalid`	Input	1	
`s_axil_wready`	Output	1	
`s_axil_bresp`	Output	`[1:0]`	
`s_axil_buser`	Output	`[UW-1:0]`	
`s_axil_btrace`	Output	1	
`s_axil_bloop`	Output	`[LW-1:0]`	
`s_axil_bvalid`	Output	1	
`s_axil_bready`	Input	1	
`fub_axil_awaddr`	Output	`[AW-1:0]`	
`fub_axil_awprot`	Output	`[2:0]`	
`fub_axil_awlock`	Output	1	
`fub_axil_awuser`	Output	`[UW-1:0]`	
`fub_axil_awtrace`	Output	1	
`fub_axil_awloop`	Output	`[LW-1:0]`	
`fub_axil_awmpam`	Output	`[MW-1:0]`	
`fub_axil_awmecid`	Output	`[EW-1:0]`	
`fub_axil_awnsaid`	Output	`[NW-1:0]`	
`fub_axil_awvalid`	Output	1	
`fub_axil_awready`	Input	1	
`fub_axil_wdata`	Output	`[DW-1:0]`	

Port	Direction	Width	Description
`fub_axil_wstrb`	Output	`[DW/8-1:0]`	
`fub_axil_wuser`	Output	`[UW-1:0]`	
`fub_axil_wpoison`	Output	`[PW-1:0]`	
`fub_axil_wvalid`	Output	1	
`fub_axil_wready`	Input	1	
`fub_axil_bresp`	Input	`[1:0]`	
`fub_axil_buser`	Input	`[UW-1:0]`	
`fub_axil_btrace`	Input	1	
`fub_axil_bloop`	Input	`[LW-1:0]`	
`fub_axil_bvalid`	Input	1	
`fub_axil_bready`	Output	1	
`cfg_monitor_enable`	Input	1	Enable monitoring
`cfg_error_enable`	Input	1	Enable error detection
`cfg_timeout_enable`	Input	1	Enable timeout detection
`cfg_perf_enable`	Input	1	Enable performance monitoring
`cfg_compl_enable`	Input	1	Enable completion packets
`cfg_threshold_enable`	Input	1	Enable threshold packets
`cfg_debug_enable`	Input	1	Enable debug packets
`cfg_timeout_cycles`	Input	`[15:0]`	Timeout threshold in MICROSECONDS (1 us tick), despite the name

Port	Direction	Width	Description
cfg_freq_sel	Input	[3:0]	counter_freq_invariant LUT index
cfg_latency_threshold	Input	[31:0]	Latency threshold for alerts
cfg_axi_pkt_mask	Input	[15:0]	Drop mask for packet types
cfg_axi_err_select	Input	[15:0]	Error select for packet types
cfg_axi_error_mask	Input	[15:0]	Individual error event mask
cfg_axi_timeout_mask	Input	[15:0]	Individual timeout event mask
cfg_axi_compl_mask	Input	[15:0]	Individual completion event mask
cfg_axi_thresh_mask	Input	[15:0]	Individual threshold event mask
cfg_axi_perf_mask	Input	[15:0]	Individual performance event mask
cfg_axi_addr_mask	Input	[15:0]	Individual address match event mask
cfg_axi_debug_mask	Input	[15:0]	Individual debug event mask
cfg_cg_enable	Input	1	Enable clock gating
cfg_cg_idle_count	Input	[CG_IDLE_COUNT_WIDTH-1:0]	Idle cycles before gating
cfg_addr_check_enable	Input	1	
cfg_addr_range_e	Input	[(N_ADDR_RANGES	

Port	Direction	Width	Description
nable		> 0 ? N_ADDR_RANGES : 1)-1:0]	
cfg_addr_filter_enable	Input	1	
cfg_addr_filter_low	Input	[AW-1:0]	
cfg_addr_filter_high	Input	[AW-1:0]	
monbus_valid	Output	1	Monitor bus valid
monbus_ready	Input	1	Monitor bus ready
busy	Output	1	
active_transactions	Output	[7:0]	Number of active transactions
error_count	Output	[15:0]	Total error count
transaction_count	Output	[31:0]	Total transaction count
cg_gating	Output	1	Gated clock is stopped
cg_idle	Output	1	No activity observed
cfg_conflict_error	Output	1	Configuration conflict detected
cfg_start_event_sel	Input	[2:0]	
cfg_end_event_sel	Input	[2:0]	
cfg_start_trigger	Input	1	
cfg_end_trigger	Input	1	
cfg_window_force_close	Input	1	
window_active	Output	1	
window_cycles	Output	[31:0]	
perf_prod_cycles	Output	[31:0]	
perf_bp_cycles	Output	[31:0]	

Port	Direction	Width	Description
perf_starv_cycles	Output	[31:0]	
perf_idle_cycles	Output	[31:0]	
perf_beat_count	Output	[31:0]	
perf_byte_count	Output	[63:0]	
perf_burst_count	Output	[31:0]	

17.4 AXI5-Lite Optional Signal Groups

Eight groups, each gated by its own ENABLE_* parameter. A group contributes to the packed SKID payload only when enabled.

Group	Parameter	Signals on this module	Width
USER	ENABLE_USER	AxUSER, WUSER, BUSER, RUSER	USER_WIDTH
TRACE	ENABLE_TRACE	AxTRACE, BTRACE, RTRACE	1
LOOP	ENABLE_LOOP	AxLOOP, BLOOP, RLOOP	LOOP_WIDTH
MPAM	ENABLE_MPAM	AxMPAM	MPAM_WIDTH
MECID	ENABLE_MECID	AxMECID	MECID_WIDTH
NSAID	ENABLE_NSAID	AxNSAID	NSAID_WIDTH
POISON	ENABLE_POISON	WPOISON, RPOISON	one bit per 64 data bits
LOCK	ENABLE_LOCK	AxLOCK	1

(Only the channels this module carries are present; a read module has no W or B channel, so it has no WPOISON or BUSER.)

17.4.1 With every group disabled, this IS the AXI4-Lite module

The packed payload width of a fully-disabled build equals its AXI4-Lite counterpart's, channel for channel:

Payload	AXI4-Lite	AXI5-Lite, groups off	AXI5-Lite, groups on
`ARSize`	35	35	75
`RSize`	34	34	43
`AWSize`	35	35	75
`WSize`	36	36	41
`BSize`	2	2	10

(at `AXIL_ADDR_WIDTH = AXIL_DATA_WIDTH = 32` and the default group widths.)

That equivalence is what `val/amba/test_axil5_master_rd.py` relies on when it drives AXI4-Lite RTL with AXI5-Lite BFMs: with no groups enabled an AXI5-Lite interface *is* an AXI4-Lite interface, so the same testbench binds to either.

17.4.2 It transports; it does not interpret

MPAM, MECID, NSAID, LOOP and TRACE are carried end to end unmodified. POISON is carried, never generated and never checked. LOCK is carried with no exclusive-access monitor behind it. Those behaviours belong to the endpoints on either side, and nothing in this module implements them.

A disabled group's OUTPUT is driven to zero rather than left dangling, so an integrator who disables a group downstream of one that enables it sees a defined value instead of X.

17.5 Notes for the monitored variants

The monitor does not observe the optional groups. `axi_monitor_filtered` has no ports for MPAM, MECID, NSAID, TRACE, LOOP or POISON, so it sees exactly what it sees on AXI4-Lite: handshakes, addresses, responses and timing. It does not check MPAM/MECID/NSAID consistency and does not validate POISON.

`ACLK_MHZ` is not decoration. It builds the microsecond tick LUT in `counter_freq_invariant`. Leave it at the 100 MHz default on a 90 MHz part and every microsecond-denominated timeout is wrong, silently.

`NUM_BANKS` > 1 on a WRITE monitor requires `USE_WDATA_ORDER_Q = 1`. `axi_monitor_trans_mgr` fails elaboration otherwise; the error names the combination.

A filter parameter only decides whether the logic is SYNTHESISED. A build that sets `ADDR_FILTER_ENABLE` but leaves `cfg_addr_filter_enable` low filters nothing and looks broken. The parameter and the runtime port are both required.

17.6 Verification

`val/amba/test_axil5_slave_wr_mon_cg.py` drives this module with the AXI5-Lite BFMs and **every optional group enabled** – `TBClasses/axil5` sets the group widths in `COMPONENT_KWARGS` to mirror the RTL defaults. A BFM configured differently from its DUT is a bind failure, which is the loud version of the mistake.

The testbench class is the AXI4-Lite one with the component factories swapped, so every phase, check and randomizer sweep has a single definition and a fix to the AXI4-Lite flow reaches this module automatically.

17.7 Related Modules

- axil5_master_rd
- axil5_master_rd_cg
- axil5_master_rd_mon
- axil5_master_rd_mon_cg
- axil5_master_wr
- axil5_master_wr_cg
- `axil4_slave_wr_mon_cg` – the AXI4-Lite counterpart
- AXI4-Lite modules

17.8 Navigation

AXI5-Lite index | AMBA overview